

Enhanced Agentic AI SDLC System - Setup Guide

Prerequisites

Before setting up the system, ensure you have the following installed:

- **Python 3.8+** (recommended: Python 3.11)
- **Node.js 16+** (for frontend components)
- **Git** (for version control)
- **Docker** (optional, for containerized deployment)

Quick Setup

1. Navigate to Project Directory

```
cd ~/JOBS
```

2. Run Setup Script

```
./scripts/setup.sh
```

3. Configure Environment

```
# Copy environment template
cp PREREQUISITES/.env.example .env

# Edit with your settings
nano .env
```

4. Start the System

```
./scripts/start_system.sh
```

Manual Setup (Alternative)

Step 1: Create Virtual Environment

```
python3 -m venv venv
source venv/bin/activate
```

Step 2: Install Dependencies

```
pip install -r PREREQUISITES/requirements.txt
```

Step 3: Initialize Database

```
python -c "  
from sqlalchemy import create_engine  
from MODELS.database import Base  
engine = create_engine('sqlite:///./sdlc_system.db')  
Base.metadata.create_all(bind=engine)  
print('Database initialized successfully')  
"
```

Step 4: Start API Server

```
cd API  
uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

Access Points

After successful setup, you can access:

- **API Documentation:** <http://localhost:8000/docs>
- **API Health Check:** <http://localhost:8000/health>
- **Frontend** (if available): <http://localhost:3000>

Environment Configuration

Required Environment Variables

```
# Database  
DATABASE_URL=sqlite:///./sdlc_system.db  
  
# API Configuration  
API_HOST=0.0.0.0  
API_PORT=8000  
SECRET_KEY=your-secret-key-here  
  
# AI Services  
OPENAI_API_KEY=your-openai-api-key  
OPENAI_MODEL=gpt-4  
  
# ChromaDB  
CHROMA_DB_PATH=./chroma_db  
CHROMA_COLLECTION_NAME=sdlc_documents
```

Optional Configuration

```
# Redis (for caching)
REDIS_URL=redis://localhost:6379

# PostgreSQL (alternative to SQLite)
DATABASE_URL=postgresql://user:password@localhost/sdlc_db

# Development settings
DEBUG=true
LOG_LEVEL=INFO
```

Docker Setup

Using Docker Compose

```
# Build and start all services
docker-compose up -d

# View logs
docker-compose logs -f

# Stop services
docker-compose down
```

Manual Docker Build

```
# Build image
docker build -t sdlc-system .

# Run container
docker run -p 8000:8000 -v $(pwd):/app sdlc-system
```

Testing the Setup

1. Run Validation Script

```
python TESTS/validation_script.py
```

2. Run Unit Tests

```
pytest TESTS/ -v
```

3. Test API Endpoints

```
# Health check
curl http://localhost:8000/health

# Create a project
curl -X POST "http://localhost:8000/projects/" \
  -H "Content-Type: application/json" \
  -d '{"name": "Test Project", "description": "Testing setup"}'
```

Troubleshooting

Common Issues

1. Port Already in Use

```
# Find process using port 8000
lsof -i :8000

# Kill the process
kill -9 <PID>
```

2. Permission Denied on Scripts

```
chmod +x scripts/*.sh
```

3. Python Module Not Found

```
# Ensure virtual environment is activated
source venv/bin/activate

# Reinstall dependencies
pip install -r PREREQUISITES/requirements.txt
```

4. Database Connection Error

```
# Check if database file exists
ls -la *.db

# Reinitialize database
rm -f sdlc_system.db
python -c "from MODELS.database import Base, engine;
Base.metadata.create_all(bind=engine)"
```

Getting Help

1. Check the logs in `logs/` directory
2. Run the validation script for detailed diagnostics
3. Ensure all environment variables are set correctly
4. Verify all dependencies are installed

Next Steps

After successful setup:

1. **Explore API Documentation:** Visit <http://localhost:8000/docs>
2. **Read Architecture Guide:** See `DOCS/architecture.md`
3. **Configure Agents:** Check `AGENTS/crew_manager.py`
4. **Set up RAG System:** Configure `RAG/rag_system.py`
5. **Add Your Data:** Use the document upload endpoints

Development Workflow

Starting Development

```
# Activate environment
source venv/bin/activate

# Start in development mode
./scripts/start_system.sh
```

Making Changes

1. Edit code in respective folders
2. API changes: Restart uvicorn server
3. Frontend changes: Hot reload should work automatically
4. Database changes: Create and run migrations

Testing Changes

```
# Run specific tests
pytest TESTS/test_api.py -v

# Run all tests
pytest TESTS/ -v

# Validate system
python TESTS/validation_script.py
```

Production Deployment

Using Docker

```
# Production docker-compose
docker-compose -f docker-compose.prod.yml up -d
```

Manual Production Setup

1. Use PostgreSQL instead of SQLite
2. Set up Redis for caching
3. Configure proper logging
4. Set up reverse proxy (nginx)

5. Use environment-specific configuration
6. Set up monitoring and health checks

Need help? Check the troubleshooting section or refer to the comprehensive documentation in the `DOCS/` folder.